

B.Sc. (Hons) in Software Development



Ollscoil
Teicneolaíochta
an Atlantaigh

Atlantic
Technological
University

Watchtower

By
Michael McDonagh

April 26, 2026

Final Report

Department of Computer Science & Applied Physics,
School of Science & Computing,
Atlantic Technological University (ATU), Galway.

Contents

1	Introduction	2
1.1	The Landscape of Modern Monitoring	2
1.2	The Need for Predictive Intelligence	3
1.3	Introducing Watchtower	3
2	Methodology	5
2.1	Software Development Methodology	5
2.2	Research Methodology	6
3	Technology Review	7
3.1	The Programming Language	7
3.2	Choosing the right Database!	11
3.2.1	Standalone Database Performance	11
3.2.2	Performance When Paired with Redis	12
3.2.3	Write and Read Trade-off	13
3.2.4	Final Decision	13
3.3	Component Based Architecture	14
3.4	Integrating with Machine Learning	14
3.4.1	Model Design	15
3.4.2	ONNX Runtime	15
3.4.3	Limitations	15
4	System Design	16
4.1	Centralised API and Management Hub	17
4.2	Metric Agents and Remote Access	17
4.3	Data Persistence and Storage	18
4.4	Authentication and Security	19
4.5	Machine Learning and Risk Assessment	19
4.6	Automated Remediation Engine	20
4.7	Containerisation	21
4.8	Administrator Features	21

4.8.1	Accounts	21
4.8.2	Roles	22
4.8.3	Profiles	23
4.9	Tickets	24
4.10	Backups	24
4.11	Session Management	25
5	System Evaluation	27
5.1	Ease of Use	27
5.1.1	Docker	27
5.1.2	First Time Setup	27
5.2	Resource Efficiency	28
5.3	Scalability and Microservice Architecture	28
5.4	Remediation	28
5.5	Machine Learning and Predictive Analysis	29
6	Conclusion	30
6.1	What Was Built	30
6.2	Challenges	31
6.3	Limitations and Future Work	31
6.4	Final Thoughts	32
7	Appendix	33

List of Figures

2.1	Trello Kanban board used to manage development tasks.	6
3.1	Memory usage comparison for the 42nd Fibonacci sequence calculation.	9
3.2	CPU utilisation across different programming languages.	10
3.3	Concurrency and parallel processing efficiency comparison.	10
4.1	System Architecture.	16
4.2	Accounts Management Interface	22
4.3	Roles and Permission Management Interface	23
4.4	Profiles Interface	23
4.5	Ticketing Interface	24
4.6	Backups Interface	25
4.7	Sessions Interface	26

List of Tables

3.1	Performance Summary: CPU, Concurrency, and Memory Usage across Languages	9
3.2	Standalone Database Benchmark Results (1000 sequential ops, 10,000 concurrent reads)	11
3.3	Benchmark Results: Databases Paired with Redis (dual-write, reads from Redis)	12
3.4	Write Throughput: Standalone vs Paired with Redis	13
3.5	Concurrent Read Throughput: Standalone vs Paired with Redis . .	13

Chapter 1

Introduction

In the early stages of the internet, managing server infrastructure was more manual. This was because much of the managing was done in-house in most cases by the same company who had their services running on those servers. Monitoring these servers was straightforward as an administrator could assess hardware health through physical observation or by executing simple command-line queries on a single machine. However, during the early 2000's there began a new rise in how servers are managed, primarily with the launch of AWS in 2006 [1] and pre-release of Google Cloud in 2008 before launching in 2011 [2]. The advent cloud service providers such as Amazon Web Services and Google Cloud Platform transformed the server from a static piece of hardware into a dynamic resource.

In the modern DevOps era, the single server model has been replaced by distributed architectures and microservices. Applications today are rarely hosted on a single machine, instead they are comprised of dozens or even thousands of interconnected **Docker** [3] containers spread across various geographical regions. While this shift has provided unprecedented scalability and uptime, it has led to a new crisis. While infrastructure becomes more abstracted and complex, the ability for a human operator to maintain a mental map of system health diminishes. This evolution has made automated, centralised monitoring no longer a luxury but a necessity. As noted in recent AIOps research, the transition from reactive, threshold-based alerting to proactive service management is critical for reducing downtime and distinguishing valid incidents from background noise in complex cloud environments [4].

1.1 The Landscape of Modern Monitoring

As the need for oversight grew, a massive industry emerged to provide tools to observe servers on a large scale. These tools are designed to aggregate logs,

metrics and traces into a single dashboard. However, a critical analysis of the current market reveals a significant divide. On one side, there are legacy open source tools, which while powerful, often suffer from archaic user interfaces and a steep learning curve that requires a significant time investment to master. On the other side are "Software as a Service" (SaaS) giants like Datadog [5] and New Relic [6].

While these proprietary platforms offer polished user experiences, they introduce two primary issues that motivated the creation of Watchtower. The first is "Complexity Fatigue" where these platforms are built to serve Fortune 500 companies with infinite budgets, resulting in much bloat where the essential metrics are buried under hundreds of proprietary modules the average user will never require. The second issue is the resource tax as many commercial monitoring agents are built using high-level, heavy runtimes that consume a non-negligible amount of the hosts CPU and RAM. For a developer running small virtual private servers, the act of monitoring the system can ironically become the primary cause of system instability.

1.2 The Need for Predictive Intelligence

Traditional monitoring has historically been reactive. A system is configured with threshold alerts, an example of such is a webhook is sent if CPU usage exceeds a certain amount. While useful, this approach is fundamentally flawed because it only informs the administrator after the system is already under duress. In a high-stakes production environment, a 90% CPU spike might lead to a total system lockup before a human can log in to intervene.

The next logical step in the evolution of system management is the integration of Machine Learning. By utilising historical data, a system should be able to identify the core of a failure, by following the specific sequence of metric fluctuations that typically precede a failure. Despite the buzz surrounding Artificial Intelligence, very few accessible, open-source tools actually implement ML in a way that is usable for the average developer. This gap in the market provided a clear opportunity for a new kind of system, one that is simple to use as a basic dashboard but with an intelligent predictive tool.

1.3 Introducing Watchtower

Watchtower was conceived as a response to these challenges. It is an open-source, multi-server management and remediation system designed with three core pillars in mind: Speed, Extensibility and Compatibility. The system provides a

centralised "Main Server" that orchestrates the deployment of ultra-lightweight "Metric Agents" across a fleet of managed servers.

To ensure the system does not interfere with the hosts it monitors, these agents are written in **Go** [7] and target a minimal memory footprint of roughly *9MB*. Beyond simple monitoring, Watchtower integrates an Automated Remediation Engine that allows users to define custom logic that when certain metrics meet the criteria for a threshold, specific tasks can be executed such as executing commands on the server or webhooks. On top of all of this, Watchtower implements a Machine Learning risk assessment by leveraging **ONNX** [8] runtime which can assess in real time the metrics of a server and give a score back to how it closely resembles a past incident.

Chapter 2

Methodology

2.1 Software Development Methodology

The software development of Watchtower followed an Agile-inspired approach. Given the projects focus on the three core pillars of Speed, Extensibility and Compatibility, it was essential to build the system in modular increments. The process began with the development of the centralised REST API in **Go** [7], which served as the Main Server. Once the core communication layer was established, the Metric Agents were developed and refined to ensure they remained ultra-lightweight, with a targeted memory footprint of approximately *9MB*.

To manage the workload throughout development, a Kanban board was maintained in Trello. Tasks were broken into cards and moved across columns as work progressed, giving a clear visual picture of what was in progress, what was blocked and what had been completed. This made it straightforward to prioritise features and avoid losing track of work across different components of the system. The Trello board is shown in Figure 2.1.

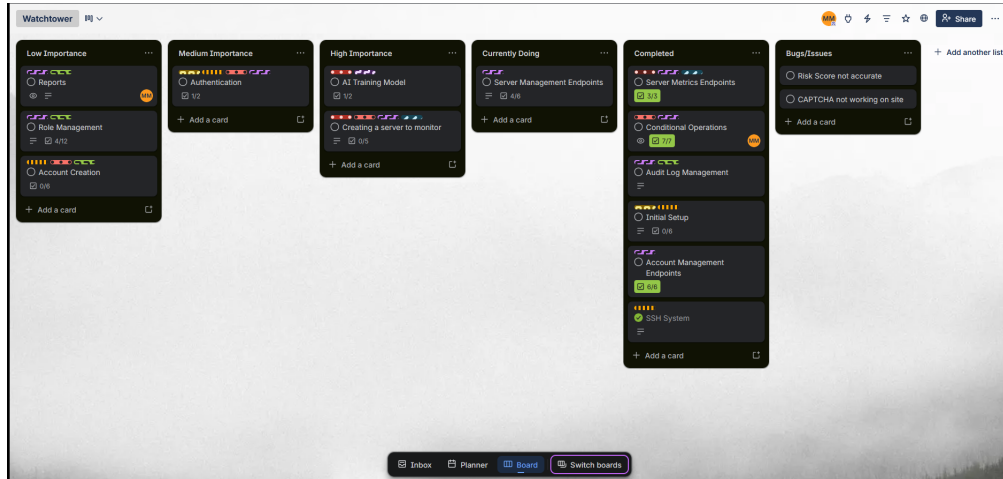


Figure 2.1: Trello Kanban board used to manage development tasks.

Testing was conducted using a Component Based Architecture, where each element was isolated within **Docker** [3] containers. This allowed for rapid prototyping and ensured that the software remained compatible across various hardware architectures like *x86* and *ARM*. This iterative cycle of building, containerising and testing allowed for the successful integration of complex features like the Automated Remediation System and Machine Learning risk assessment.

2.2 Research Methodology

The research methodology for this dissertation was primarily applied and comparative. The initial phase involved a critical analysis of the modern monitoring landscape, identifying a seismic shift from physical servers to distributed microservices and Docker containers. Secondary research was conducted on existing market leaders such as Datadog [5] and New Relic [6] to identify gaps, specifically focusing on Complexity Fatigue and high resource consumption.

To address the need for predictive intelligence, the research focused on the implementation of Machine Learning through supervised learning. Historical metric data was pulled from a PostgreSQL database to identify the specific sequences of fluctuations that precede system failures. Furthermore, the research explored the use of the ONNX (Open Neural Network Exchange) [8] runtime to allow the Go-based API to execute predictive model without the heavy overhead typically associated with Python environments. This research was then validated during the System Evaluation phase by measuring the tools resource efficiency and ease of use compared to traditional, reactive monitoring methods.

Chapter 3

Technology Review

The thought process for Watchtower began with three simple pillars, Speed, Extensibility and Compatibility. These were the pillars I followed when deciding how to structure the project, as without a proper foundation, a project will reach its limits much sooner.

3.1 The Programming Language

As with all pieces of software, the choice of programming language will have a wide impact on everything to come after that decision. The programming language of choice for this project was **Go** [7]. The decision to choose this as the primary language for the project comes down to the three pillars I mentioned earlier.

Unlike many programming languages and their respective frameworks like JavaScript with NodeJS or Deno, Ruby with Rails, Python with Flask for creating Web interfaces or API's, they all have something in common... they are slow! Well compared to the likes of C, C++ and many other lower level languages. It isn't as common nowadays to write web software or API's in lower level languages due to the rise of the ones I mentioned because they are much simpler and as a result offer a much larger eco system, tooling and support. This is something that had stuck with me when thinking which language do I want to write my program in. The language that I found to have the speeds I need as well as the eco system was in fact **Go** [7]!

A bit of background about the language is that it was developed by Google back in 2007 by Robert Griesemer, Rob Pike, and Ken Thompson. The reason it was developed was due to criticisms of other programming languages at Google but didn't want to trade off the benefits of those languages such as simplicity and usability. Google needed a language that was both easy to type, having runtime efficiency like C and be great for high-performance multi-processing. The language

also offers low memory usage and built-in concurrency, which is something I found to be necessary for a project like this. According to the original design philosophy of the language, Go was specifically engineered to address the complexities of modern networked systems and multicore processors, providing a robust architecture that remains simple and adaptable as software scales [9].

Programs written in **Go** [7] are compiled to a single file binary that can be executed on any hardware without needing to install extra packages and programs as with Node, Flask or Rails. It is also more easily setup than languages like C/C++ on a fresh server without needing to install dependent binaries and other dependencies. I have examples of the simplicity of the Go programming language vs the likes of Java. The example showcases a simple Goroutine (see Listing 3.1) and a Java thread (see Listing 3.2), both offer concurrency in their languages.

Listing 3.1: Go Goroutine Example

```
1 package main
2 import "fmt"
3
4 func main() {
5     go fmt.Println("Hello from a goroutine")
6 }
```

Listing 3.2: Java Thread Example

```
1 public class Main {
2     public static void main(String[] args) {
3         Thread thread = new Thread(() -> {
4             System.out.println("Hello from a thread");
5         });
6         thread.start();
7     }
8 }
```

The way it is designed to fit with my project is both the API and the Metric Agents are written in **Go** [7]. The main benefits of using **Go** [7] in this way is that the low memory usage will ensure that for monitoring servers that the memory used by the program does not interfere with the servers metrics its monitoring, preventing skewing! As for the API, since the system is designed to handle hundreds of servers, Go's built in concurrency became a huge benefit, as compared to languages like Java that is generally used for enterprise systems, the memory usage of Go is very little.

To put this into perspective, I conducted several tests on languages such as Java, JavaScript, Python, and **Go** [7], comparing their use of concurrency, CPU, and memory usage. This was performed by having each language calculate the

42nd Fibonacci number. These tests measured the total memory consumption (see Figure 3.1), the impact on CPU resources (see Figure 3.2), and the efficiency of each language when handling these operations in parallel (see Figure 3.3).

The table for Performance Summary 3.1 showcases the results of my findings. Although it might seem Java could be a clear winner having bet **Go** [7] at CPU bound tasks, Java did have to consume $5x$ more memory to be 28% faster for CPU tasks and 25% faster for concurrency, which when working with hundreds of servers, or even thousands, it will add up, and eventually lead to running out of memory and needing to upgrade, which is costly.

Table 3.1: Performance Summary: CPU, Concurrency, and Memory Usage across Languages

Language	CPU (ms)	Concur (ms)	Mem (MB)
Go	3,415.5	705.5	9.8
Python	107,799.0	27,379.8	190.6
JavaScript	7,232.1	3,848.6	1,101.3
Java	2,442.4	544.8	49.1

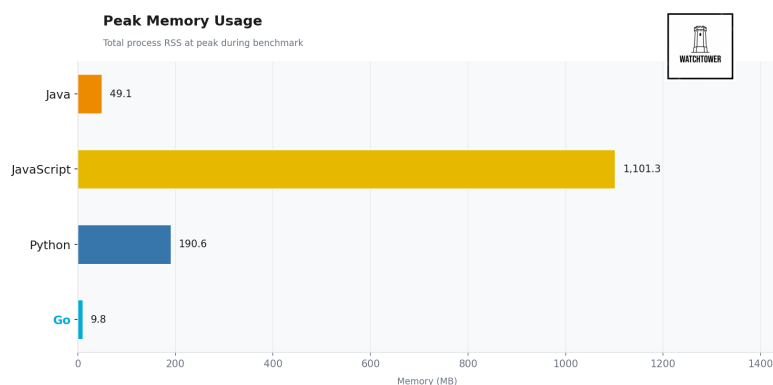


Figure 3.1: Memory usage comparison for the 42nd Fibonacci sequence calculation.

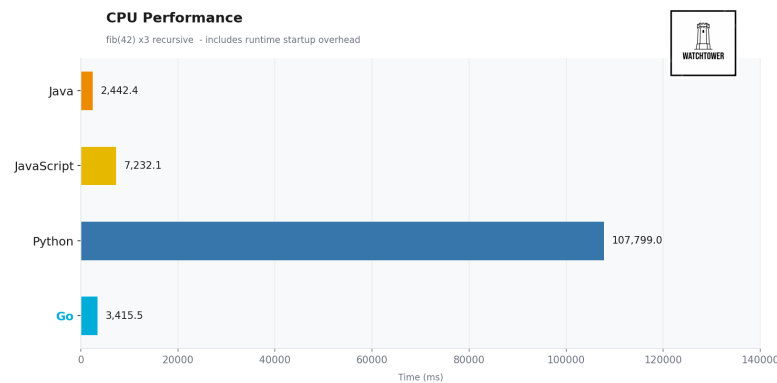


Figure 3.2: CPU utilisation across different programming languages.

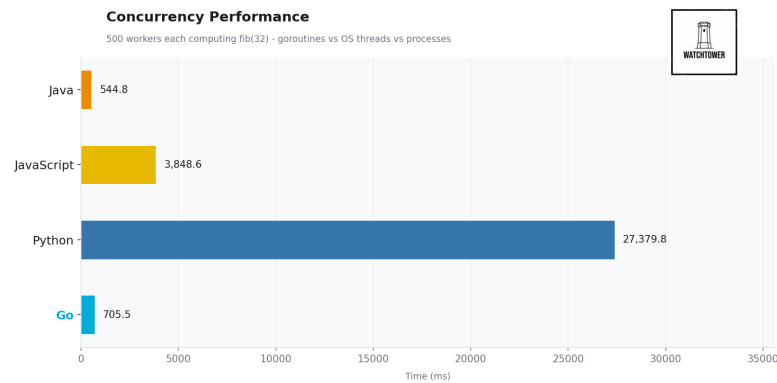


Figure 3.3: Concurrency and parallel processing efficiency comparison.

The performance disparity between these two concurrency models comes down to how they interact with the host operating system. Traditional languages like Java rely on OS-level threads. Each OS thread typically reserves a large, fixed block of memory for its stack and requires an expensive context switch handled by the operating system kernel. In contrast, Go implements user-space threads known as goroutines. A goroutine starts with a microscopic stack footprint of just a few kilobytes, which grows and shrinks dynamically as needed. The Go runtime includes its own highly efficient scheduler that maps thousands of these lightweight goroutines onto a small pool of actual OS threads. This architecture completely bypasses the heavy kernel-level context switching, allowing Watchtower to handle thousands of concurrent metric processing tasks with a fraction of the memory overhead required by an enterprise Java environment.

3.2 Choosing the right Database!

When doing my research into what database would work best for a project like Watchtower it was important that it succeeded in a few different areas and in order to find the right database I had to perform some testing on these different key areas.

- **Write Speed** - How quickly can a record be inserted? For a monitoring system receiving a continuous stream of metrics from multiple servers, slow writes create a bottleneck that delays how quickly data reaches the system.
- **Read Speed** - How quickly can the latest data be retrieved? For a live dashboard, every millisecond of read latency is visible to the user. Both sequential reads (one client at a time) and concurrent reads (many clients simultaneously) needed to be considered.
- **Scalability under concurrent load** - A monitoring system is not used by one person at a time. Multiple users may be viewing live dashboards simultaneously while servers are continuously writing metrics. The database must handle this concurrent pressure without degrading performance.

3.2.1 Standalone Database Performance

To establish a baseline, each database was benchmarked in isolation - no **Redis** [10] involved. Each database performed 1,000 sequential writes, 1,000 sequential reads, and then 10,000 concurrent reads across 50 goroutines simultaneously, which simulates multiple users polling a live dashboard at the same time.

Table 3.2: Standalone Database Benchmark Results (1000 sequential ops, 10,000 concurrent reads)

Database	Write total	Write/op	Seq read total	Seq read/op	Conc total	Throughput
PostgreSQL	177ms	177ms	72ms	72ms	195ms	51,411 reads/s
MySQL	1252ms	1252ms	118ms	118ms	229ms	43,725 reads/s
MongoDB	177ms	177ms	205ms	205ms	345ms	29,001 reads/s

The results from Table 3.2 reveal a clear picture. For write speed, **PostgreSQL** [11] and **MongoDB** [12] were equal, both completing 1,000 inserts in 177ms at 177ms per operation. **MySQL** [13] was significantly slower at 1,252ms, over *seven* times slower than the other two. This is an important finding for a monitoring system where metrics are being written continuously from multiple servers - a slow write path creates a backlog that delays how current the data is.

For read speed, **PostgreSQL** was the strongest performer. It completed *1,000* sequential reads in just *72ms* (*72ms* per read) and handled *10,000* concurrent reads in *195ms*, achieving a throughput of *51,411* reads per second. **MongoDB**, while competitive on writes, was noticeably slower on reads at *205ms* per sequential read and only *29,001* reads per second under concurrent load. This difference is due to **PostgreSQL**'s structured schema and indexing, which allows it to locate the latest record far more efficiently than **MongoDB**'s document scan [14]. **MySQL** sat in between but its poor write performance already made it a weaker candidate overall.

Based on the standalone results alone, **PostgreSQL** [11] was the clear choice as the primary persistent database. It was the fastest at both writing and reading, and handled concurrent load better than either alternative [14].

3.2.2 Performance When Paired with Redis

The next stage of testing introduced **Redis** into the benchmark. In this scenario, every write was sent to both the database and **Redis** simultaneously, but all reads were served exclusively from **Redis**. This mirrors the exact design used in Watchtower for live metrics.

Table 3.3: Benchmark Results: Databases Paired with **Redis** (dual-write, reads from **Redis**)

Scenario	Write total	Write/op	Seq read total	Seq read/op	Conc total	Throughput
Redis + PostgreSQL	812ms	812ms	282ms	282ms	557ms	17,958 reads/s
Redis + MySQL	2005ms	2005ms	274ms	274ms	519ms	19,269 reads/s
Redis + MongoDB	825ms	825ms	289ms	289ms	530ms	18,865 reads/s

As shown in Table 3.3, pairing any database with **Redis** produces consistent read performance regardless of which underlying database is used. All three Redis-paired scenarios completed sequential reads in approximately 274-289ms and achieved concurrent read throughput between 17,958 and 19,269 reads per second. This consistency makes sense, in this design, reads never touch the database at all, so the choice of database has no impact on read performance. **Redis** is the only component serving read requests.

The write performance reflects the cost of dual-writing. **Redis + PostgreSQL** and **Redis + MongoDB** completed writes in a similar time (812ms and 825ms respectively), while **Redis + MySQL** was again the slowest at 2,005ms due to **MySQL**'s slower insert performance.

3.2.3 Write and Read Trade-off

Table 3.4 confirms that adding **Redis** always reduces write throughput, since every metric must be written to two systems instead of one. PostgreSQL’s write throughput drops from 5,640 to 1,231 writes per second, a 4.6x reduction. This is a trade-off that was consciously accepted in Watchtower, because writes happen on the server side in the background and a small delay is tolerable. What is not tolerable is a slow read path on the live dashboard.

Table 3.4: Write Throughput: Standalone vs Paired with Redis

Database	Standalone	With Redis	Difference	Winner
PostgreSQL	5,640 writes/s	1,231 writes/s	4.6x	Standalone
MySQL	799 writes/s	499 writes/s	1.6x	Standalone
MongoDB	5,640 writes/s	1,213 writes/s	4.7x	Standalone

Table 3.5 shows that standalone **PostgreSQL** outperformed **Redis** on concurrent reads in this benchmark. This result requires important context. The benchmark was executed on a single local machine, meaning all three systems, the application, the database, and **Redis** all shared the same hardware with no network latency between them. In this environment, **PostgreSQL**’s connection pool is extremely efficient. In a real production deployment such as Watchtower, each component runs in a separate Docker container and communicates over a network. Every database read in that environment involves a network round trip, which adds latency that accumulates significantly under concurrent load. **Redis**, being an in-memory store, absorbs that latency far better than a disk-backed relational database. The local benchmark therefore understates the advantage **Redis** provides in production.

Table 3.5: Concurrent Read Throughput: Standalone vs Paired with Redis

Database	Standalone	With Redis	Difference	Winner
PostgreSQL	51,411 reads/s	17,958 reads/s	0.3x	Standalone
MySQL	43,725 reads/s	19,269 reads/s	0.4x	Standalone
MongoDB	29,001 reads/s	18,865 reads/s	0.7x	Standalone

3.2.4 Final Decision

The benchmark results, taken together, justified the choice of **PostgreSQL** as the primary persistent database and **Redis** as the live read layer. **PostgreSQL** was the strongest standalone performer across all three key metrics, write speed,

read speed, and concurrent throughput, making it the right choice to store data durably. **Redis** was introduced not to replace **PostgreSQL** but to sit in front of it for time-sensitive reads, shielding the database from being queried on every live dashboard poll across every connected client. Historical data such as audit logs and account information continues to be read directly from **PostgreSQL**, where a slower response is acceptable. This separation of concerns, writing to both, reading live data from **Redis** and historical data from **PostgreSQL**, is the design that Watchtower implements.

3.3 Component Based Architecture

When designing how my architecture will operate, containerisation of the components was a important factor. My choice of containerisation was **Docker** [3] due to it being industry standard, easy to use and familiar with majority of people in the software world. Each component such as the WebUI, Command Line Interface (CLI), API, Database, Metric Agents, were all **Docker** [3] images. The benefit of using Docker goes beyond the scope of this project but the main thing it does, it just *works*. Regardless of the host operating system be it Windows, MacOS or any of the GNU/Linux distributions, you can be guaranteed that it will run on that hardware.

Choosing to use **Docker** [3], even for the Metric Agents was an important choice, as servers being monitored might operate on different hardware architectures like *x86* or *ARM*, as well as different Operating Systems.

3.4 Integrating with Machine Learning

One of the more ambitious objectives for Watchtower was the integration of machine learning to provide predictive analysis on server health. Existing monitoring tools like Datadog do offer anomaly detection, but it is typically a black box feature behind a paid tier. The goal here was to build something that could be understood, customised and shipped as part of the system itself.

The approach taken was supervised learning. Historical server metrics stored in **PostgreSQL** [11] are used to train a model that learns the relationship between a server's current state and what its resource usage is likely to look like. Rather than simply alerting when a threshold is crossed, the model analyses patterns in the data leading up to past issues and produces a risk score based on how similar the current state is to those known problematic states.

3.4.1 Model Design

The model takes *16* input features representing the current state of a server, including CPU usage, memory, disk, swap, network connections and uptime. From these inputs it predicts *five* output values: memory allocated, memory allocations, total disk usage, used disk and free disk. These outputs are compared against the actual current values to produce a similarity percentage, which acts as the risk score surfaced to the user on the dashboard.

Before being passed into the model, the input features are normalised using a pre-computed scaler. The outputs are then inverse-transformed back to their original scale using a corresponding output scaler. This scaling step is important because without it the model would be biased towards features with larger numerical ranges, such as disk usage in bytes, over features like CPU percentage.

3.4.2 ONNX Runtime

A key design decision was how to run the model inside a Go-based API without introducing a Python dependency. Training a model in Python is standard, but deploying it in a non-Python environment is a challenge most implementations sidestep by running a separate Python service. For Watchtower this was not acceptable as it would add another component to the stack, increase resource usage and go against the core pillar of simplicity.

The solution was to export the trained model to the **ONNX** [8] format (Open Neural Network Exchange). **ONNX** [8] is an open standard for representing machine learning models that is runtime and language agnostic. Using the `onnxruntime_go` library, the **Go** [7] API can load and run inference on the **ONNX** [8] model directly, with no Python process involved at any point. This means the predictive analysis runs inside the same API binary that handles all other requests, keeping the architecture clean and the resource overhead low.

3.4.3 Limitations

The main limitation of this approach is that the model is trained before the software is released rather than continuously on the user's own infrastructure. This means the model is trained on general server behaviour patterns and may not immediately reflect highly specific or unusual workloads on a given server. It will produce a meaningful risk score in most cases, but the accuracy improves the more the server's behaviour resembles patterns seen in training. A future improvement would be to allow the model to be retrained locally on the user's own historical data, which would make the predictions far more tailored to their specific environment.

Chapter 4

System Design

The system design of Watchtower puts a huge focus on the user experience, from installation all the way to managing servers. The architecture is designed to be robust, production-grade and capable of monitoring multiple servers from a single central dashboard.

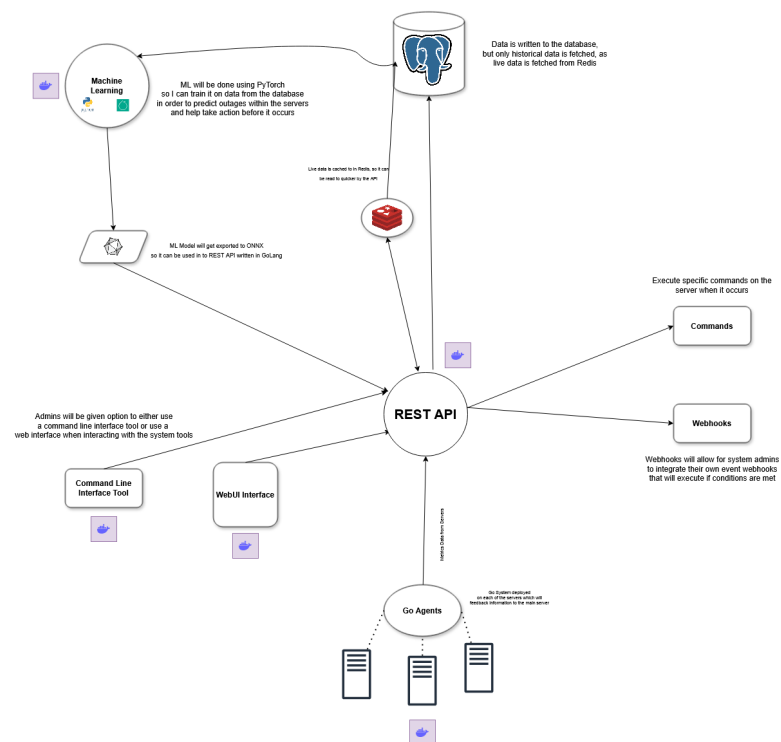


Figure 4.1: System Architecture.

4.1 Centralised API and Management Hub

At the core of the architecture lies the **REST API** [15], built using the **Gin framework** [16] in **Go** [7] and exposed on port *8080* under the `/api/v1` prefix. **Gin** [16] was chosen specifically because of its middleware system, which made it straightforward to layer authentication, permission checking and logging on top of every route without repeating code. According to its documentation, **Gin** [16] is up to *40* times faster than Martini, a comparable **Go** [7] framework, which aligned well with the performance requirements of the project. This is the central piece of software that every other component in the system talks to. The Web UI, the CLI and the Metric Agents all communicate exclusively through this API. Nothing in the system talks directly to the database or to another component, everything goes through the API.

On startup the API does a number of things before it is ready to accept requests. It initialises the **JWT** [17] authentication system, connects to **PostgreSQL** [11] and runs the schema to ensure all tables exist, starts a background ping loop that checks all registered servers every *60* seconds, starts a health checker that runs every *10* seconds, initialises the **Redis** [10] connection and starts the backup scheduler. This startup sequence means the system is fully operational the moment the API is running, with no manual configuration steps required after the initial setup.

The API uses **JWT** (JSON Web Tokens) [17] for authentication. A JWT is an open standard that defines a compact, self-contained way to securely transmit information between parties as a JSON object [17]. Each token consists of three parts, a header, a payload and a signature, which together allow the server to verify both the integrity and the origin of every request without needing to look up a session in the database on every call. Every request to a protected route must include a valid Bearer token in the **Authorization** header. The middleware validates the token signature, loads the permissions for that user's role from the database and attaches them to the request context. From that point on, individual routes can check for specific permissions without needing to query the database again. Sessions are also validated on every request, so if an administrator revokes a session the user is locked out immediately on their next request.

4.2 Metric Agents and Remote Access

The Metric Agents are lightweight programs written in Go that run on each server being monitored. Their only job is to collect system metrics at regular intervals and send them back to the main server API. The metrics collected include CPU usage, memory usage, swap, disk usage, SSH connections, HTTP and HTTPS

connection counts and system uptime. Since they are written in Go and compiled to a single binary, the agents have a very small footprint. In practice the observed memory usage of a running agent is around *9MB*, which is important as the agent should have as little impact on the server it is monitoring as possible.

Installing an agent on a new server is done directly from the Watchtower dashboard. When a server is added, the user provides the IP address, SSH username, SSH port and a private key. From there the API uses that SSH connection to remotely install and start the agent on the target server without the user ever needing to log into it directly. The installation progress is streamed back to the dashboard in real time, so the user can watch the installation happen line by line. Once installed, the agent logs are also accessible from the dashboard, again streamed live, which makes debugging much easier if something goes wrong.

The dashboard also provides a remote console feature. Using the same SSH connection details, an administrator with the appropriate permissions can execute commands on a managed server directly from the browser. This was an important feature to include as it means the operator never needs to leave the Watchtower interface to deal with an issue on a server.

4.3 Data Persistence and Storage

When a Metric Agent sends a payload to the API, the data is written to two places at the same time. It is inserted into **PostgreSQL** [11] for long term persistent storage, and it is also pushed into a **Redis** [10] list using the server's ID as the key. The **Redis** [10] entry is set to expire after *24* hours, which keeps the memory usage controlled since old live data does not need to be kept in memory indefinitely.

When the dashboard requests metrics for a server, the API checks **Redis** [10] first. If valid data is found there it is returned immediately without touching **PostgreSQL** [11]. If **Redis** [10] is empty or the data is invalid, the API falls back to querying **PostgreSQL** [11]. This design means that live dashboard views are served entirely from memory in the normal case, which is significantly faster, while **PostgreSQL** [11] acts as the source of truth for all historical data including audit logs, account information, machine learning input and anything else where speed is not the primary concern.

The choice to write to both simultaneously rather than writing only to **PostgreSQL** [11] and reading through **Redis** [10] as a cache was intentional. It means that even if Redis is unavailable or has been recently restarted, the API can fall back to PostgreSQL without any data loss. The two stores serve different purposes rather than one being a replica of the other.

4.4 Authentication and Security

Security was an important consideration throughout the design given that Watchtower has access to SSH keys, can execute remote commands and holds sensitive infrastructure data. The authentication system is built around JWT tokens with a session layer on top. When a user logs in a session record is created in **PostgreSQL** [11]. Every subsequent request validates both the **JWT** [17] signature and checks that the session is still marked as active in the database. If an administrator revokes a session from the sessions page, the next request made by that user will be rejected even if their token has not expired yet.

The decision to utilise JSON Web Tokens (JWT) was driven by the distributed nature of the Watchtower architecture. Traditional monolithic applications often rely on stateful, cookie-based sessions, which require the server to store session data in memory or query a database on every single request to verify the user. For a system processing continuous metric payloads from dozens of remote agents, this database lookup would quickly become a severe bottleneck. Since a JWT is cryptographically signed and self-contained, it holds all necessary authorisation data within the token payload itself. When a Metric Agent sends data, the Gin middleware simply verifies the cryptographic signature using the server's secret key. If the signature is valid, the request is authorised locally in memory without a single database query, drastically improving the throughput of the central API.

Every significant action performed through the API is recorded to an audit log. This includes things like creating a server, updating an account, triggering a backup and so on. Each audit log entry captures the user who performed the action, the type of action, the resource it was performed on, the IP address of the request and the user agent. This gives administrators a full trail of what happened and who did it, which is useful for both security and accountability in team environments.

Permissions are enforced at the route level using middleware. Each route that requires a specific permission checks the permissions attached to the current user's role before allowing the request through. This means that even if a user somehow obtains a valid token for an account with a restricted role, they cannot access routes their role does not have permission for.

4.5 Machine Learning and Risk Assessment

As established in the Technology Review, the decision was made to use a supervised learning model exported to **ONNX** [8] format and run natively inside the Go API using `onnxruntime_go`. This section covers how that integration fits into the overall system design.

The risk score is exposed through the `/risk/server/:id` route. When this endpoint is called, the API takes the most recent metrics for that server, applies the pre-computed input scaler to normalise the 16 feature values, passes them through the **ONNX** [8] model, and inverse-transforms the outputs back to their original scale. The resulting similarity score between the predicted and actual values is returned and displayed on the dashboard as a health indicator for that server.

Because the model runs inside the same API process rather than in a separate service, there is no additional network call involved in producing a risk score. The inference happens in the same request cycle as any other API call, which keeps the latency low and the architecture simple.

4.6 Automated Remediation Engine

The remediation system is designed around the idea that every server is different. Not every server is running the same services or has the same thresholds that matter, so the system needs to be flexible enough to handle any combination of conditions and actions.

Every managed server can have any number of remediation groups. Within each group is a set of conditions that can be combined using AND and OR logic, along with a set of actions that will be executed when those conditions are met. Conditions can check any of the collected metrics against a threshold, so for example a condition could check that CPU usage has exceeded 90% AND that the number of HTTP connections is above 1000. The actions that can be triggered include sending a webhook to an external service, rebooting the server, executing an arbitrary command on the server and more.

This flexibility means that the remediation engine can be used in very different ways depending on what the server is doing. A server running a website might have a remediation group that watches connection counts and fires a webhook to notify an on-call team if traffic spikes unexpectedly. A general purpose server might have a group that reboots it automatically if memory usage stays above a critical threshold for too long. The system does not try to make these decisions for the user, it just gives them the tools to set up whatever response makes sense for their infrastructure.

To illustrate the power of this flexibility, consider a scenario where a managed server is subjected to a distributed denial-of-service attack. A rigid monitoring tool might simply alert the administrator that CPU usage is high. Using Watchtower's remediation engine, an operator can construct a highly targeted defence mechanism. They could create a remediation group containing two distinct conditions: a CPU utilisation sustained above 95 percent AND an HTTP connection

count exceeding 10,000 active sockets. If both conditions evaluate to true, the engine can execute a predefined action, such as triggering an external webhook to a cloud firewall provider to instantly activate strict rate limiting. This transforms Watchtower from a passive observation dashboard into an active infrastructure defence system, capable of mitigating threats before a human operator even opens the alert notification.

4.7 Containerisation

Every component of Watchtower is containerised using **Docker** [3]. This includes the API, the Web UI, the databases and even the Metric Agents. The reason for this is straightforward: it means the software works the same regardless of what operating system or hardware it is running on. Whether the main server is running Ubuntu, Debian, or any other Linux distribution, the **Docker** [3] images will behave identically.

For the Metric Agents this was particularly important because the servers being monitored can vary significantly in their hardware and operating system. By packaging the agent as a **Docker** [3] image it can be deployed to any server that has **Docker** [3] installed, whether it is running on *x86* or *ARM* hardware, without needing to compile or configure anything on the target system.

The tradeoff of this approach is that **Docker** [3] adds a layer of complexity for users who are not familiar with it. However the installation process is designed so that a regular user only needs to have **Docker** [3] installed on their main server. The agents are deployed automatically from the dashboard and **Docker** [3] is installed on the managed servers as part of that process.

4.8 Administrator Features

Watchtower is designed to be used by teams, not just individual users. This meant that a proper account and permission system was essential from early on in the design process. Without it, there would be no meaningful way to control who can do what, which is a serious concern when the software has the ability to reboot servers, execute commands and access sensitive infrastructure data.

4.8.1 Accounts

The first interaction any operator will have with Watchtower is during the initial setup of the main server, where they are prompted to create the primary administrator account. This account has full access to everything in the system. From

that point on, the accounts page is available where further accounts can be created, updated or deleted as needed. 4.2

Every account is tied to a role and a profile. On its own the account is fairly straightforward, it is the role that determines what that account can and cannot do, and the profile that gives the user control over their own personal details. This separation keeps the account management clean and gives administrators a clear way to control access without having to manage permissions on a per-user basis.

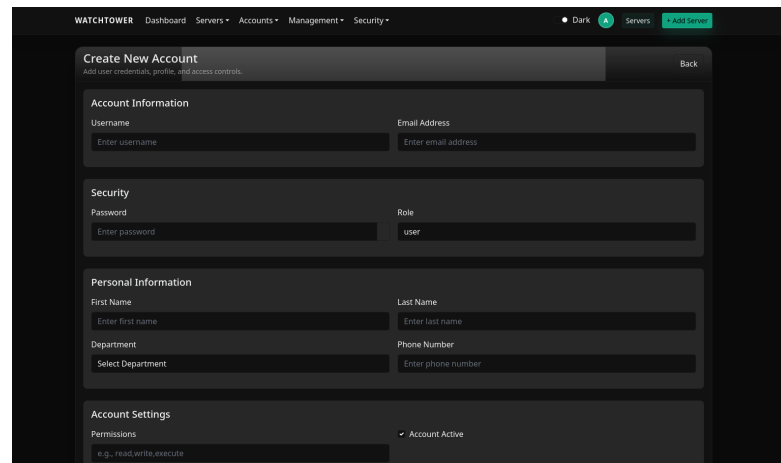


Figure 4.2: Accounts Management Interface

4.8.2 Roles

When managing infrastructure at any meaningful scale it is important to separate responsibility. Not everyone who uses the system should have the same level of access. A junior team member monitoring servers should not have the same permissions as a senior engineer who can manage accounts or change system settings.

The way roles are implemented in Watchtower is that every account is assigned a role at the point of creation. The role defines which pages and features that account has access to. If a permission is not granted to a role, the associated pages will not be visible or accessible to any account holding that role.

Creating and managing roles is straightforward. An administrator can set the role name, assign it a colour for easy identification in the interface, and toggle individual permissions on or off. A good example of this in practice is the default role called *operator*, shown in Figure 4.3, which grants access to server management, backups and sessions, but deliberately excludes account management. This ensures that operators can do their day to day work without having the ability to create new accounts or change permissions, which reduces the risk of accidental or unauthorised changes.

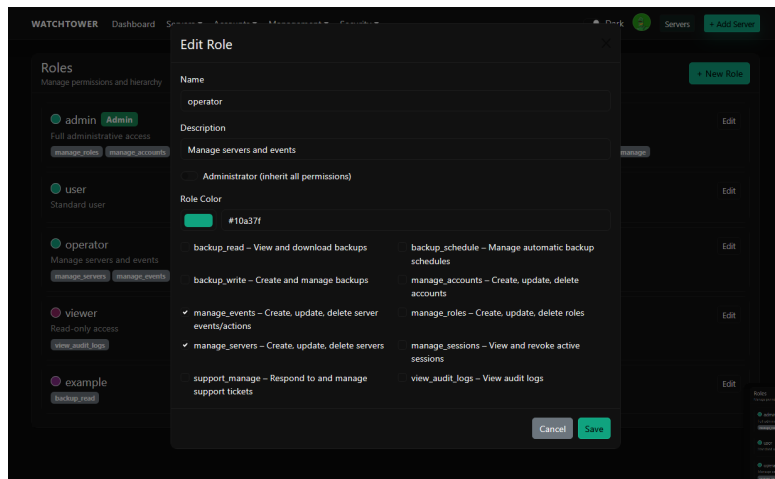


Figure 4.3: Roles and Permission Management Interface

4.8.3 Profiles

Profiles are the part of the account system that is accessible to all users, not just administrators. This is where a user can manage their own personal details such as their email address, display name and password, as well as cosmetic preferences like their profile image and profile colour. The profile colour is used throughout the interface to make it easier to identify who is who at a glance, which is a small but useful detail when multiple people are working in the system at the same time. This can be seen in Figure 4.4.

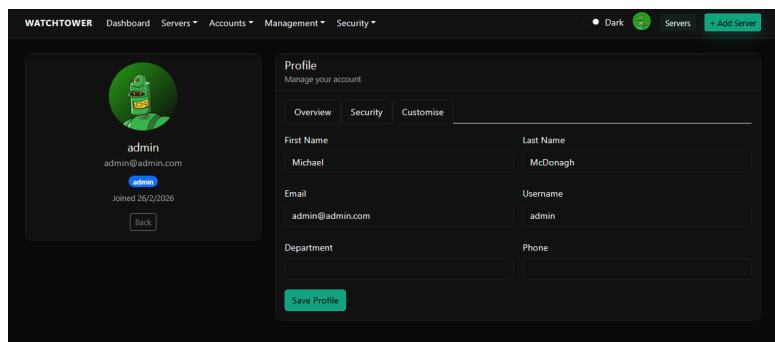


Figure 4.4: Profiles Interface

4.9 Tickets

Watchtower was always intended to be used by teams, and teams need a way to communicate about issues. This led to the implementation of a simple internal ticketing system. Any user can open a ticket by setting a title and describing the issue. From there, a team member with the ticket permission can view the open tickets, investigate the issue and work towards resolving it.

Once an issue is resolved the ticket can be archived to keep the active list clean. If the issue comes back it can be reopened rather than requiring a new ticket to be created. When the ticketing permission is combined with other permissions in the roles system, it allows certain team members to have exactly the right combination of access to both investigate and fix the issues being reported. This can be seen in Figure 4.5.

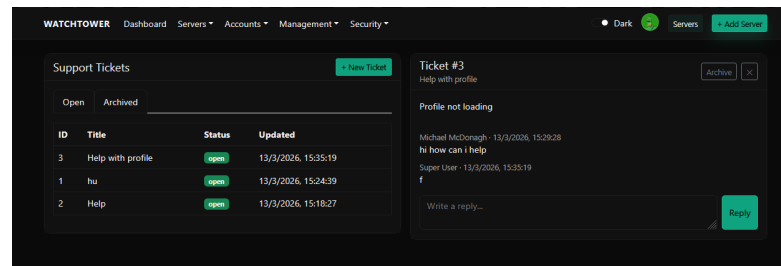


Figure 4.5: Ticketing Interface

4.10 Backups

The data collected by Watchtower has value beyond just the live dashboard. Historical metrics, audit logs and configuration data all need to be protected against accidental loss. To address this, Watchtower includes an automated backup system that runs on a schedule, by default every *12* hours, though this interval can be adjusted in the server settings to suit the needs of the deployment.

Outside of the scheduled backups, an administrator can also trigger a manual backup at any point using the Backup Manually button. Backups are packaged as a `.zip` file containing all data from the **PostgreSQL** [11] database, which can be downloaded and stored externally. This gives operators confidence that even in the event of a catastrophic failure, the data can be restored. This can be seen in Figure 4.6.

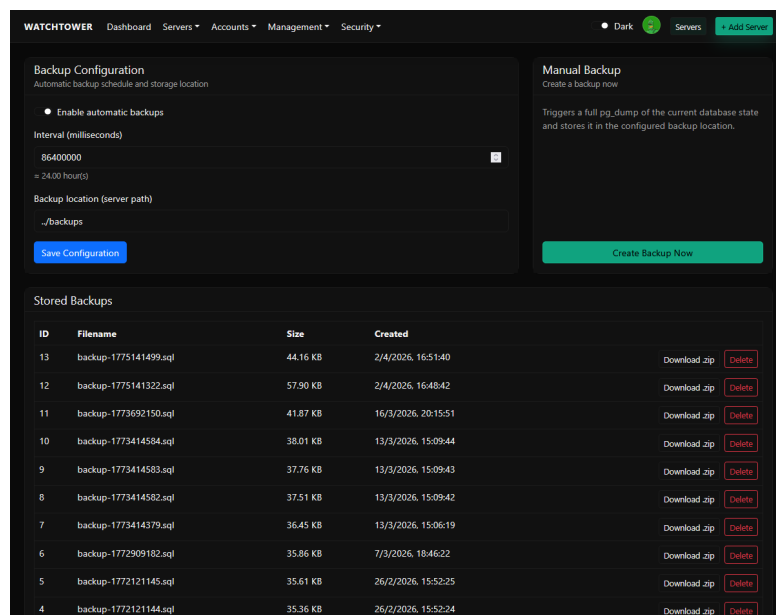


Figure 4.6: Backups Interface

4.11 Session Management

When a system has access to sensitive infrastructure, knowing who is currently logged in and from where is an important security concern. Watchtower includes a session management page where every active login session is displayed, showing the IP address of the device and the user agent, which includes information about the browser and operating system being used.

If an administrator notices a session that looks suspicious or belongs to a user who should no longer have access, they can revoke that session immediately, invalidating the login token and forcing that user to log in again. This is a straightforward but important feature, as it gives administrators a real-time view of who has access to the system at any given moment and the ability to act on it quickly. This can be seen in Figure 4.7.

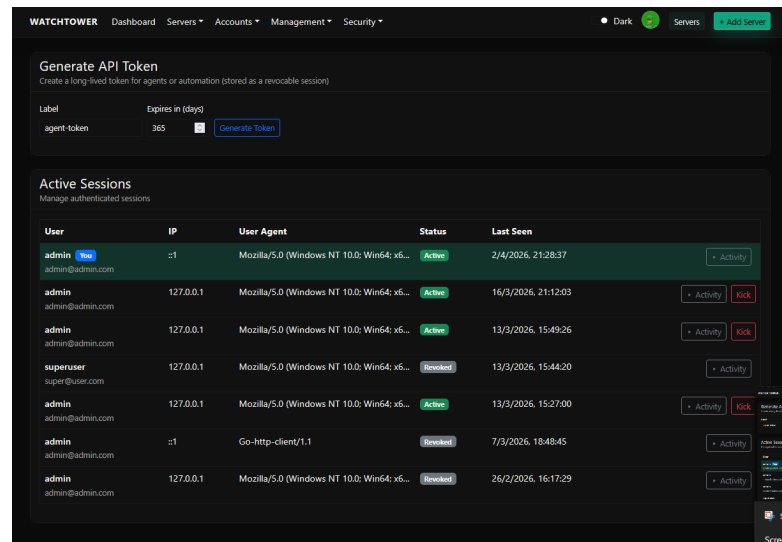


Figure 4.7: Sessions Interface

Chapter 5

System Evaluation

Watchtower has been developed with many features, each of which keep in line with the three pillars mentioned earlier in this dissertation. This chapter evaluates the system against the objectives set out in the introduction, discussing the strengths and weaknesses observed throughout development.

5.1 Ease of Use

One of the main objectives was to ensure a near seamless user experience. This experience is designed from the installation and setup of the main server, all the way to the one-click installation and updating of managed servers.

5.1.1 Docker

There are a few different cross-overs with this objective and the containerisation objective, but in essence containerisation allowed for a seamless installation of Watchtower regardless of system operating system and hardware. Although there are many pros of using **Docker** [3] for the project, it does come with a slight learning curve, but that is only limited to people who wish to contribute. For regular users the only knowledge of **Docker** [3] required is installing it on their main server, as the servers being managed will have **Docker** [3] automatically installed.

5.1.2 First Time Setup

The first time setting up the software had the goal of being as simple as possible. This was something that needed much more thought as it is more of a chicken and egg situation. If there is no account, how can the API start to create one? This

was a question that came up a lot and I had to address it. The resolution turned out to be more straightforward than expected. When the API first starts up, the user is prompted to enter the credentials for the administrator account via the CLI. The benefit of this approach is that it is simple and requires no prior configuration, however it does come with a trade-off in that the credentials are passed through the CLI, meaning a compromised system could potentially log those keys.

5.2 Resource Efficiency

When it comes to system monitoring tools, as laid out in my objectives, it is important that the software has little interference with the hardware it is running on. In order to achieve this efficiency across the board from the main server to monitored servers, it was important that the software had the right foundation. This came down to the choice of language, which was **Go** [7]. Some of its many strengths include low memory usage, where the observed memory usage of an agent at runtime was roughly *9MB*. The weakness of the metric agents is the frequent polling, which has the potential for a spike in CPU or network I/O if the monitored server has limited bandwidth.

5.3 Scalability and Microservice Architecture

The choice of separating components moved the software away from a traditional monolithic architecture to a more microservice based approach. The separation of the API, database and **Redis** [10] means that even in the case where the web dashboard goes down, the system will still operate as normal. This does increase the complexity in setup for the average user if they decide to host each service on a different machine. In cases where a metric is not showing on a graph, the user may need to trace back through multiple services to find the source of the issue, which increases the time and complexity required to troubleshoot.

5.4 Remediation

Automated remediation was an important feature that needed to be implemented along with being entirely customisable. This was achieved by allowing each server to have groups containing conditions, paired with AND and OR logic, that when met will execute a list of actions such as webhooks, reboots and custom scripts. The main limitation of a feature like this is speed. If a CPU spike causes a server to become unresponsive, there is a question of whether the system can execute the

remediation actions fast enough to prevent a crash. In most cases the answer is yes, but in situations of extreme load it may not always be guaranteed.

5.5 Machine Learning and Predictive Analysis

The integration of a pre-trained machine learning model was a critical objective to differentiate Watchtower from existing tools like Datadog.

The system successfully uses supervised learning to analyse historical data from **PostgreSQL** [11]. By tracing metrics leading up to past server failures, the model produces a risk score based on similarity to known problematic states.

A significant strength of this implementation is the use of **ONNX** [8] (Open Neural Network Exchange) runtime. By exporting the model to this format, the **Go** [7]-based API can execute complex predictive analysis without the heavy resource overhead typically associated with Python-based machine learning environments. This standard for interoperability allows the system to decouple the inference engine from original training dependencies, ensuring high portability and performance across different environments [18].

Chapter 6

Conclusion

Watchtower was built to address a clear gap in the server monitoring space. The existing tools available to developers are either too complex, too expensive or too resource-heavy to be practical for anyone managing a small to medium number of servers. The goal from the beginning was to build something that was fast, simple to use and capable enough to go beyond basic threshold alerting, without becoming the very problem it was designed to solve.

Looking back at the three core pillars that guided the design, Speed, Extensibility and Compatibility, the system delivers on all three. The choice of **Go** [7] as the primary language for both the API and the Metric Agents kept the resource footprint low, with agents running at around *9MB* of memory in practice. The microservice architecture built on **Docker** [3] means the system runs consistently regardless of the underlying hardware or operating system. The Automated Remediation Engine gives users the flexibility to define their own logic for any situation their infrastructure might present, rather than being locked into a fixed set of responses.

6.1 What Was Built

The system that was delivered includes a REST API built in **Go** [7] using the **Gin** [16] framework, a web dashboard, lightweight Metric Agents that can be deployed remotely over SSH, a dual-database storage layer using **Redis** [10] for live data and **PostgreSQL** [11] for persistence, a role-based access control system, an automated remediation engine with AND/OR condition logic, a machine learning risk assessment system running **ONNX** [8] inference natively in **Go** [7], a ticketing system, session management, audit logging and automated backups.

The benchmark testing carried out as part of this project provided empirical evidence for the data storage design decisions. **PostgreSQL** [11] proved to be

the strongest standalone database across write speed, sequential read speed and concurrent read throughput when compared against **MySQL** [13] and **MongoDB** [12]. The decision to pair it with **Redis** [10] for live metric reads was justified not by the local benchmark results alone, but by the production context where every read crosses a network boundary between containers and **Redis's** [10] in-memory access absorbs that latency far better than a disk-backed database under concurrent load.

6.2 Challenges

The most technically challenging part of the project was the machine learning integration. Getting a Python-trained model to run inside a **Go** [7] API without a Python dependency required exporting to **ONNX** [8] format and using the `onnxruntime_go` library. This was not something I had used before and required a significant amount of research and testing to get working reliably. The result however is that the predictive analysis runs inside the same API process with no additional infrastructure required, which was the goal.

The first-time setup flow was another area that required more thought than expected. Getting the system to a point where it was easy to install for a new user, while also being secure from the first interaction, involved working through the chicken and egg problem of how to create the first administrator account before there are any accounts to authenticate with. This was easily resolved by having the admin details passed into the API upon first setting up, not needed if rebooting or starting up the API again.

Although **Go** [7] was heavily used in this project, I had only began programming in the language 2 months prior, so learning the different strengths of the language, along with the weaknesses was a great learning experience.

6.3 Limitations and Future Work

The primary limitation of the current machine learning architecture is its reliance on a static, pre-trained model. Currently, the ONNX runtime evaluates incoming server metrics against a fixed set of weights established prior to deployment. Since it is trained on general server behaviour patterns rather than the specific infrastructure of the user, the risk scores are meaningful but may lack precision for highly specific or unusual workloads. While highly effective for generalised anomaly detection, this approach cannot fully adapt to the evolving baseline behaviour of a maturing server environment.

A significant avenue for future development lies in transitioning to a continuous,

adaptive learning pipeline. Implementing a lightweight, local retraining mechanism would allow the model to periodically update its weights based on a server's unique historical data. This would make the predictions far more tailored to the user's environment, allowing Watchtower to organically learn the difference between a genuine resource leak and a scheduled, expected spike in routine batch processing. Ultimately, this adaptive approach would reduce false-positive alerts and vastly improve the overall reliability of the automated remediation engine.

Regarding the remediation engine, the current implementation is highly flexible but executes actions synchronously. In situations of extreme server load, this could mean a remediation action fires too slowly to prevent a system failure. A necessary future improvement would be to give users granular control over action priority and asynchronous execution timing to ensure critical interventions occur immediately.

Finally, on the monitoring side, the current scope focuses strictly on system-level metrics. A natural extension of this platform would be to support application-level metrics. Allowing users to monitor application specific data like database query times, request latency, and error rates alongside the existing infrastructure metrics would transform Watchtower into a fully comprehensive observability suite.

6.4 Final Thoughts

Watchtower set out to be a monitoring tool that a developer could actually use without spending days configuring it or paying for a subscription. What was built goes well beyond a basic dashboard. It is a system that can install agents remotely, remediate issues automatically, predict problems before they happen and give a team the tools they need to manage infrastructure together. The three pillars of Speed, Extensibility and Compatibility held up throughout the entire development process and shaped every significant design decision that was made.

Chapter 7

Appendix

- **GitHub Repo** - <https://github.com/McDonaghMichael/Watchtower>
- **Demonstration Video** - <https://youtu.be/cD0AnEx1Bic>

Bibliography

- [1] Amazon Web Services. Our origins. <https://aws.amazon.com/about-aws/our-origins/>, 2024. Accessed: 02-04-2026.
- [2] Pluralsight. The history of Google Cloud Platform. <https://www.pluralsight.com/resources/blog/cloud/history-google-cloud-platform>, 2023. Accessed: 02-04-2026.
- [3] Docker, Inc. Accelerate deployment with Docker. <https://www.docker.com/>, 2026. Accessed: 04-04-2026.
- [4] Adedamola Abiodun Solanke. Aioops evolution: From reactive monitoring to predictive service management in hybrid cloud environments. *Frontiers in Computer Science and Information Technology (FCSIT)*, 3(1):37–59, 2022.
- [5] Datadog. Cloud monitoring as a service. <https://www.datadoghq.com/>, 2026. Accessed: 02-04-2026.
- [6] New Relic. The ai-powered digital intelligence platform. <https://newrelic.com/>, 2026. Accessed: 02-04-2026.
- [7] The Go Authors. The Go programming language. <https://go.dev/>, 2026. Accessed: 04-04-2026.
- [8] ONNX. Open neural network exchange. <https://onnx.ai/>, 2026. Accessed: 04-04-2026.
- [9] Rob Pike. Go at google: Language design in the service of software engineering. Keynote talk given at the SPLASH 2012 conference, 2012.
- [10] Redis. The real-time data platform. <https://redis.io/>, 2026. Accessed: 04-04-2026.
- [11] The PostgreSQL Global Development Group. PostgreSQL: The world’s most advanced open source relational database. <https://www.postgresql.org/>, 2026. Accessed: 04-04-2026.

- [12] MongoDB, Inc. The developer data platform. <https://www.mongodb.com/>, 2026. Accessed: 04-04-2026.
- [13] Oracle Corporation. MySQL: The world’s most popular open source database. <https://www.mysql.com/>, 2026. Accessed: 04-04-2026.
- [14] Brandon (Outerbase). Postgres vs. MySQL. <https://dev.to/outerbase/postgres-vs-mysql-14cp>, 2025. Accessed: 04-04-2026.
- [15] Lokesh Gupta. What is REST. <https://restfulapi.net/>, 2026. Accessed: 04-04-2026.
- [16] The Gin Authors. Gin web framework — documentation. <https://gin-gonic.com/en/docs/>, 2026. Accessed: 04-04-2026.
- [17] Auth0. Introduction to JSON web tokens. <https://jwt.io/introduction>, 2026. Accessed: 04-04-2026.
- [18] Silin Zhao. Improving portability and interoperability of deep-learning-workloads using onnx. Master thesis, Georg-August-Universität Göttingen, 2025.